

Greedy algorithms

Lecturer: Yuri Faenza

1 Introduction

Informally speaking, in greedy algorithms, we have a complex algorithmic decision to make. Instead of finding the “globally optimal” decision, we take a more myopic approach and take a decision that looks “locally optimal”, i.e., promising at a more superficial level. Among the many reason why we want to do this, some relevant ones are:

- We may acknowledge that the complexity of the problem is beyond our abilities, hence we will never be able to find a globally optimal decision, so we settle for a locally optimal one.
- We may want to solve the problem fast, either because the size of the instance is huge (hence any complex algorithm will require too much time), or because the motivating application we have in mind requires the output to be produced fast (think for instance of a search engine query).
- The output of the greedy algorithm may serve as a starting point and / or a benchmark for other, more complex algorithms.

In this lecture, we see many example of greedy procedures – sometimes, they work well, sometimes they don’t, and sometimes they give us the optimal solution!

2 Covering problems

Consider the following *sensor placement* problem, illustrated in Figure 1. We want to understand how polluted is a river. We have n fixed locations $i_1, \dots, i_n$ where we can place sensors: each sensor, will give us information on the amount of pollution in a radius of 1 meter around the sensor. Unfortunately, our budget is limited, and we can only place sensors in k locations: which should we choose so as to maximize the total area covered? If all the areas covered by sensors were disjoint, the problem would be trivial: any selection of k locations will be optimal. However, those areas may overlap – this makes the problem challenging.

Here is a simple greedy procedure for this problem.

-
1. Let $S = \emptyset$.
 2. **While** $|S| < k$:
 - (a) Choose $i_j \notin S$ such that the area covered by $S \cup \{i_j\}$ is maximized.
 - (b) Add i_j to S .
 3. Output S .

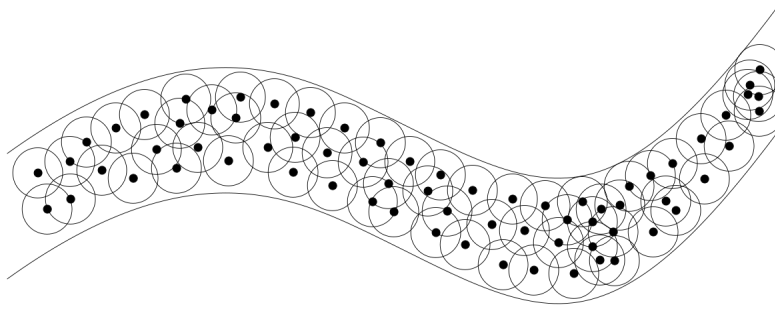


Figure 1: An instance of the sensor placement problem. In the figure, the possible locations are denoted by black dots, while the circle around each dot denotes the 1 meter radius.

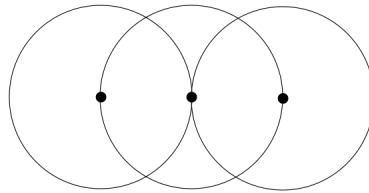


Figure 2: In the instance above, when $k = 2$, a greedy algorithm may first choose the central sensor, and then either the left or the right sensor. A better solution skips the central sensor.

It is easy to see that the algorithm may not always output the optimal solution¹, see Figure 2. We can however prove that the value of the optimal solution will not be too far away from the one output by the algorithm (the proof of this fact is beyond the scope of this lecture).

Theorem 1 *Suppose that the best choice of k sensors for the sensor placement problem covers a region of size A . Then the greedy algorithm covers an area of size at least $(1 - \frac{1}{e}) \cdot A$.*

A similar greedy procedure solves the dual of the problem seen above, choose as few sensors as possible, so that the region that is covered has area at least a given value A .

1. Let $\bar{A} = 0$, $S = \emptyset$.
2. **While** $\bar{A} < A$:
 - (a) Choose $i_j \notin S$ such that the area covered by $S \cup \{i_j\}$ is maximized.
 - (b) Add i_j to S .
 - (c) Set $\bar{A}$ to be the area covered by S .
3. Output S .

¹However, this algorithm is, in a certain well-defined sense, optimal: no algorithm whose running time is polynomial in the size of the input can beat it, in the worst-case scenario. A precise formulation of this statement and its proof are beyond the scope of these notes.

A result similar in spirit to Theorem 1 can be proved for the algorithm above.

Covering problems also appear in more abstract settings than the ones we have just seen.

Example 2 *You are setting up a company, and need to decide who to hire from a set $\{i_1, \dots, i_n\}$ of candidates. You foresee that for your everyday business, a certain number of tasks, $T_1, \dots, T_k$, will need to be performed, and for each worker, you know which task they can perform. Your budget allows you to hire only k employees. Your goal is to hire them such that the number of tasks that can be completed by a hired employee is maximized (hence, unlike the assignment / matching problem seen in a previous class, the same worker may be employed for more than one task).*

The problem from Example 2 can be formalized as the so-called *Set Covering* problem: given sets $T_1, \dots, T_k$ over points $i_1, \dots, i_n$, and a number k , choose points so that the number of sets that contain at least a chosen point is maximized. We can adapt the algorithm for the sensor placement problem to Set Covering by replacing step (a) with

(a') Choose $i_j \notin S$ such that the number of sets that contain at least an element from $S \cup \{i_j\}$ is maximized.

Example 3 *Consider the problem faced by a search engine, that we will call Chandler. When someone queries a word, Chandler wants to show them a set of k images from its database of images. In selecting the images, Chandler's goal is twofold. First, it wants to show images that are pertinent to the word that the user has searched. For instance, if the user search "jaguar", Chandler prefers to show them a picture of the animal with the same name, rather than that of a tree. Second, Chandler wants some diversification among the k images that are shown: rather than showing only pictures of the animal called jaguar, Chandler would prefer to show also show pictures of the car and of the football team with the same name.*

The problem from Example 3 is usually modelled by associating to each set S of images, a function $p(S)$ measuring how pertinent it is with the searched word (higher values of $p(S)$ denote a more pertinent collection of images), and a function $d(S)$ denoting how diversified it is (again, the higher $d(S)$, the better). Then we can use the algorithm seen for the sensor placement problem, by replacing (a) with

(a'') Choose $i_j \notin S$ such that $p(S \cup \{i_j\}) + d(S \cup \{i_j\})$ is maximized.

Results similar in spirit to Theorem 1 can be proved for the (abstract version of) problems from Example 2 and Example 3.

3 Scheduling

Suppose we are given a set of jobs $i_1, \dots, i_n$, where each job i_j has a time when it starts, denoted by s_j , and a time when it finishes, denoted by f_j . The time period between s_j and f_j is called the *execution period* of job i_j . We have a single machine for executing the jobs, and no two jobs whose execution periods overlap can both be executed. Our goal is to decide which jobs to schedule on the machine so that as many jobs as possible are executed.

For example: we have 5 jobs, denoted below by their starting and finishing times pairs: $(1, 5), (2, 4), (3, 7), (4, 6), (5, 7)$. A possible solution schedules job 2 and 5.

Consider the following greedy procedure.

-
1. Let $S = \emptyset$.
 2. If the set S' of jobs whose starting time is not earlier than the finishing time of all jobs in S is non-empty:
 - (a) Let i_j be a job from S' with earliest finishing time.
 - (b) Set $S' = S' \cup \{i_j\}$.
 3. Output S .
-

What is the rationale behind the procedure above? Suppose we want to iteratively build a set S of jobs to be scheduled, with the property that, at each step of the construction, we do not wish to schedule any job that starts before the end of the last job in S (this property is trivially true for the starting set $S = \emptyset$). Then the best way we can extend S is by choosing a job from S' that leaves as much “free” space on the machine as possible. Because of my constraint above, the free space is the one after the finishing time of the last job in S , as chosen by the algorithm. This intuition motivates the following result.

Theorem 4 *The greedy procedure gives the optimal solution to the scheduling problem above.*

4 The Travelling Salesman Problem

In the Traveling Salesman Problem, we have a set $V = \{v_1, \dots, v_n\}$ of cities, and $c : V \times V \in \mathbb{N}$ a symmetric function denoting the distance between two cities. The goal is to find a tour, i.e., an ordering of the cities, of minimum total distance, where the distance associated to the tour

$$v_1, v_2, \dots, v_{n-1}, v_n$$

is

$$c(v_1, v_2) + c(v_2, v_3) + \dots + c(v_{n-1}, v_n) + c(v_n, v_1).$$

A greedy solution can be obtained as follows (note that, unlike previous algorithms, here S is an ordered set):

-
1. Let $S = (v_1)$.
 2. **While** $|S| < n$:
 - (a) Let v be the last node of S .
 - (b) Among all nodes from $V \setminus S$, choose the node v_i that minimizes $c(v, v_i)$.
 - (c) Set $v = v_i$.
 - (d) Append v_i at the end of S .
 3. Output S .

Although faster than solving an integer program, this algorithm may output a solution whose value is far away from the optimal one. It becomes however an interesting procedure when it is used as a starting point for more sophisticated algorithms (it can, for instance, be used to produce a starting solution for a branch-and-bound based integer programming solver).

5 Minimum Spanning Tree

Consider the following problem: an electrical company wants to connect its power plant to a series of junction boxes around town. Denote the power plant by i_0 and the boxes by $i_1, \dots, i_N$. Some pairs of these locations can be connected directly – and the costs of these connections are known – while some other cannot. Can we find what is the cheapest way of realizing the overall connection?

This problem can be formalized as follows: Given a graph $G = (V, E)$ with costs $c : E \rightarrow \mathbb{N}$, find a set $E' \subseteq E$ such that:

- The graph $G' = (V, E')$ is *connected*, i.e., there is a path in G' between any two nodes of V .
- Subject to the constraint above, the total cost of edges of E' is minimum.

We are of course assuming that the input graph G is connected, else no solution exists. Moreover, notice that v_0 does not play a special role in this formulation – i.e., the optimal solution is independent of which of the nodes of G represents the power plant.

Let us observe that the optimal solution the problem above is going to be a *tree*, that is, a connected graph without a cycle. It is in fact easy to see that, from any feasible solution G' containing a cycle C , one can remove any edge of C , preserve the connectivity of the graph, and diminish the total cost. Hence, the problem above is known as the *Minimum Spanning Tree* (MST) problem.

There are many ways one can define a greedy algorithm for this problem. In this lecture, we are concerned with the following, which starts from the graph with no edges, keeps a cycleless, infeasible solution, and stops when it gives a connected graph.

-
1. Sort elements in E by increasing cost: $c(e_1) \leq c(e_2) \leq \dots \leq c(e_m)$.
 2. Set $i = 1$.
 3. Let $E' = \emptyset$.
 4. While $G = (V, E')$ is not connected:
 - (a) If $G = (V, E' \cup \{e_i\})$ does not contain a cycle, add e_i to E' .
 - (b) Set $i = i + 1$.
 5. Output $G' = (V, E')$.
-

It is easy to see that the output solution is a tree. Moreover, the following can be shown.

Theorem 5 *The greedy algorithm gives an optimal solution to the MST problem.*

Note that a tree is not very “robust”: the elimination of any of its arcs disconnects the graph. In real-world scenarios, additional edges are often added to the MST in order to make the solution robust.

6 Knapsack

The knapsack problem is a classical one in discrete optimization. It can be motivated by the following vanilla investment problem.

Example 6 *There are n investment vehicles, each coming with a required lump sum w_i to be invested today, and with an expected payoff p_i to be collected in a year. Knowing that our current budget is β , which set of investments we should choose so as to maximize the overall expected payoffs in a year?*

The problem is formalized as follows: given a set of n items $\{1, \dots, n\}$ (i.e., the investment vehicles), where to each item i are associated numbers w_i (“weight”) and p_i (“profit”), and given moreover a number β (“capacity”), choose among all sets $S' \subseteq S$ with $w(S') \leq \beta$, the one that maximizes $p(S')$.

A good greedy algorithm should not be naive! Here is a simple greedy idea, where we select items by decreasing profit.

-
1. Sort items by profit: $p(1) \geq p(2) \geq \dots \geq p(n)$.
 2. Set $S = \emptyset$
 3. For $i = 1, \dots, n$:
 - (a) If $w(S \cup \{i\}) \leq \beta$, set $S = S \cup \{i\}$.
 4. Output S .
-

This idea in general does not work very well (can you find an example where it does extremely bad?). One of the reasons is that it ignores the costs of the items. Greedy ideas that only look at the costs, ignoring the profits, work similarly bad. A more successful approach takes into account the *ratio* between profit and costs, hence replacing Step 1. with the following:

- 1'. Sort items by profit/weight ratio: $\frac{p(1)}{c(1)} \geq \frac{p(2)}{c(2)} \geq \dots \geq \frac{p(n)}{c(n)}$.